Este es el resumen de **“La Caída de Clipper: Cómo Windows Cambió la Programación Empresarial”** (canal Archivo del Futuro, 27 min). Lo hice a partir de la transcripción del video:

1. El origen: dBASE. dBASE juntaba en un solo programa tablas de datos, herramientas para consultarlas y un lenguaje parecido al inglés. Eso permitió que contadores y consultores pequeños hicieran sistemas a medida para comercios. El problema era que se trataba de un entorno interpretado, poco práctico para distribuir el programa y proteger el código.

2. La gran idea de Clipper (Nantucket). Era un “compilador para dBASE”: se seguía programando casi igual (archivos .PRG), pero el resultado era un .EXE que el cliente ejecutaba en DOS sin tener dBASE instalado. Así el programador entregaba un producto, no código fuente. El video aclara que no era código máquina puro: dentro del .EXE iba código intermedio (p-code) con una máquina virtual.

3. Summer ‘87 y su ecosistema. Esa versión se volvió el estándar. Alrededor creció todo un mundo de librerías, enlazadores, depuradores, revistas y grupos de usuarios. El trabajo diario era editar, compilar y enlazar con un archivo por lotes (.bat).

4. Los límites de DOS:

- Pantallas de texto manejadas con el teclado y datos en archivos DBF compartidos en red (Novell).
- Bloqueo de registros y riesgo de índices dañados.
- El límite de 640 KB de memoria, que obligaba a usar *overlays* con enlazadores como RTLink o Blinker.
- Las empresas no actualizaban de versión para no correr riesgos: preferían lo que ya funcionaba.

5. Clipper 5 y la compra por Computer Associates (1992). Clipper 5 trajo bloques de código y mejoras en el acceso a datos (RDD). CA siguió sacando versiones (5.2 y 5.3) y desarrolló su sucesor para Windows, **Visual Objects** (proyecto Aspen).

6. Por qué Windows lo frenó. Windows cambió el modelo de interacción: eventos, mouse, interfaz gráfica, nuevos instaladores y controladores. Visual Objects era muy ambicioso, pero obligaba a hacer una migración grande, mientras que Visual Basic, FoxPro y Delphi eran más fáciles de vender y aprender. Para el video, CA no “mató” a Clipper a propósito: el camino oficial hacia Windows simplemente dejó de ser tan sencillo como la promesa original.

7. No murió del todo. Muchos sistemas en Clipper siguieron funcionando durante años. Hoy sobreviven gracias a **Harbour** y **xHarbour** (compiladores libres compatibles que generan programas para Windows y Linux de 32 y 64 bits) y a **FiveWin** (librería gráfica).

Existen porque la lógica de negocio guardada en ese código vale más que la edad de la herramienta.

Idea central: Clipper ganó porque evitó una ruptura (te dejaba seguir usando el lenguaje de dBASE). Perdió los proyectos nuevos cuando la llegada de Windows exigió una ruptura que ya no podía evitar.

Este es el resumen de **"Por qué el mundo entero sigue dependiendo de Java"** (Archivo del Futuro). Igual que con el otro video, lo hice con la transcripción completa de lo que dice el narrador.

1. Origen (1991–1995). En Sun, James Gosling, Mike Sheridan y Patrick Naughton iniciaron el *Proyecto Green* para programar aparatos de electrónica de consumo. Hicieron un prototipo con pantalla táctil, el Star7 (1992). El lenguaje se llamó primero *Oak* y después *Java*.

2. La gran idea: la máquina virtual. El código no se compila para un procesador concreto, sino a un formato intermedio llamado *bytecode*. Ese bytecode lo ejecuta la **JVM**, que existe para cada sistema. De ahí viene el lema "escribe una vez, ejecuta en cualquier lugar", que solo funciona si hay una JVM compatible y las librerías necesarias. Además, se parecía a C/C++, pero sin punteros y con recolector de basura.

3. La web y los applets (1995). Java se lanzó junto al navegador HotJava y Netscape lo adoptó. Los *applets* eran pequeños programas que corrían dentro de las páginas. El video aclara que **JavaScript es otro lenguaje** (lo creó Brendan Eich en Netscape) y que el nombre parecido solo causó confusión. HotSpot mejoró la velocidad al compilar sobre la marcha las partes del programa que más se usan.

4. Pleito con Microsoft. Microsoft agregó a su versión de Java (Visual J++) funciones exclusivas de Windows. Sun lo demandó en 1997 y en 2001 llegaron a un acuerdo: Microsoft pagó 20 millones de dólares y perdió la licencia.

5. Celulares y empresas (1999 en adelante):

- **J2ME / Java ME:** los juegos .jar de los celulares de antes de los smartphones. Como cada modelo era distinto, los juegos a veces se veían mal.
- **Servidores:** aquí está su gran éxito, con JDBC, servlets, WebSphere, WebLogic y Tomcat. Después llegó **Spring** (2003) como alternativa más ligera.

6. Apertura y Oracle. Desde 1998 la plataforma evoluciona con el Java Community Process. Entre 2006 y 2007 Sun liberó el código como **OpenJDK**, y hoy contribuyen IBM, Amazon, Red Hat, SAP y Microsoft. **Oracle compró Sun en 2010**. En 2019 empezó a cobrar las actualizaciones comerciales de *su* JDK, pero OpenJDK sigue siendo libre (por ejemplo, con Temurin o Amazon Corretto).

7. Android. Usa el lenguaje Java, pero su propia máquina virtual (Dalvik y luego ART). Oracle demandó a Google y en 2021 la Corte Suprema de EE. UU. falló que el uso de las interfaces de Java en Android fue *uso justo*. Desde 2019, Google prioriza **Kotlin** para Android.

8. Evolución reciente:

- Java 8 (2014) agregó lambdas y streams.
- Otros lenguajes como Kotlin y Scala también corren sobre la JVM.
- *Minecraft: Java Edition* sigue siendo un ejemplo muy conocido.
- En 2013, EE. UU. recomendó desactivar Java en los navegadores por problemas de seguridad, y los applets fueron desapareciendo.
- Java 21 trajo los *hilos virtuales*, y **Java 26 (marzo de 2026) eliminó por completo la API de applets**.

Idea central: Java no sobrevivió por su lema, sino por todo lo que se construyó a su alrededor: la JVM, la compatibilidad entre versiones, el código abierto y un enorme ecosistema de librerías y servidores. Así siguió vigente aunque cambiaran sus dueños y los dispositivos donde se usa.

Este es el resumen de **"El Secreto de Ruby on Rails: la Tecnología 'Muerta' que Mueve a los Gigantes de Internet"** (Archivo del Futuro). Como en los anteriores, lo hice con la transcripción completa.

1. Ruby y Rails no son lo mismo. Ruby es un lenguaje creado por Yukihiro "Matz" Matsumoto en Japón y publicado en 1995. **Rails** es un *framework* web construido con Ruby. El video lo compara con una cocina ya montada: muchas decisiones básicas vienen tomadas, y cada decisión que te ahorras es tiempo (y sueldo) que no pagas.

2. Origen en Basecamp (2004). David Heinemeier Hansson (DHH) lo sacó del código de Basecamp, en la empresa 37signals, y lo publicó como código abierto en julio de 2004. No salió de la teoría: es lo que quedó de construir un producto real. En 2005 se hizo famosa su demo de un blog funcional hecho en unos 15 minutos.

3. Su ventaja. Se basa en **convención sobre configuración** (por ejemplo, si hay un modelo "Producto", espera una tabla "productos"), en el principio "no te repitas", en Active Record y en el *scaffolding*. No hacía más potente al servidor, sino más productivo al equipo: **5**

personas podían competir con departamentos enteros. Apple llegó a incluirlo en Mac OS X Leopard.

4. El mito de Twitter. La versión popular dice: "Twitter abandonó Rails porque no escalaba". La realidad fue más matizada:

- En 2009, las páginas de Twitter seguían hechas en Rails. Lo que reemplazaron fueron servicios internos (colas, procesos largos), que pasaron a Scala.
- En 2011 cambiaron el buscador a Java (Blender), con mejoras reales, pero solo para ese componente.
- Fue una migración por partes, no un funeral.

5. Rails sí escala, con disciplina:

- **Shopify:** un "monolito modular" de más de 2,8 millones de líneas. En vez de abandonar Rails, ordenaron mejor su código por dentro.
- **GitHub:** alrededor de 2 millones de líneas, más de 1.000 ingenieros y hasta 20 publicaciones al día.
- **Basecamp:** se reescribió en 2012 y 2015, y sigue en Rails.

La pregunta correcta, según el video, no es "¿Rails escala?", sino qué parte necesita escalar, cómo y con qué equipo.

6. Por qué perdió protagonismo. Llegaron React y las aplicaciones de página única, Node.js (JavaScript en el servidor), la nube y los microservicios. Muchos equipos pequeños copiaron arquitecturas pensadas para gigantes. Hubo además un costo humano: muchos programadores de Rails tuvieron que reaprender el oficio porque su herramienta dejó de aparecer en las ofertas de empleo, no porque dejara de funcionar.

7. Sigue vivo:

- Hotwire y Rails 8 (noviembre de 2024), que trae colas y caché sobre la propia base de datos y la herramienta Kamal para publicar aplicaciones.
- La Rails Foundation (2022) y más de 7.000 colaboradores a lo largo de su historia.
- YJIT, el acelerador de Ruby impulsado por Shopify.
- Rails 8.1 con soporte y parches de seguridad en 2026.
- En la encuesta de Stack Overflow su uso bajó de ~8 % (2019) a casi la mitad, con una leve subida en 2025.

8. El giro de la IA. La web oficial de Rails ahora se presenta como ideal para **agentes de IA**: como las convenciones dicen dónde va cada cosa, el código sería más fácil de generar y revisar y gastaría menos *tokens*. El video advierte que esto es **posicionamiento, no una ventaja comprobada**, porque no hay estudios independientes que lo demuestren.

Idea central: Rails no murió, **dejó de estar de moda**. Su caída fue relativa: tuvo límites técnicos reales, un cambio de mercado y algo de moda. Además, muchas de sus ideas las

copiaron otros frameworks, y por eso ya no parecen novedad. Hoy es una tecnología madura que sigue funcionando dentro de empresas enormes.

Este es el resumen de **“Por qué es IMPOSIBLE matar a COBOL”**. Lo hice con la transcripción completa; es más corto que los anteriores (unos 15 minutos).

1. El desprecio. Arranca con Edsger Dijkstra, ganador del premio Turing, que en 1975 escribió que COBOL “atrofia la mente” y que enseñarlo debería ser un delito. Durante décadas las universidades dejaron de enseñarlo. Aun así, sigue sosteniendo una parte enorme de los sistemas bancarios y de las transacciones del mundo.

2. Grace Hopper, la creadora. En la Segunda Guerra Mundial la Marina de EE. UU. no la quería aceptar: tenía 34 años, pesaba menos del mínimo y era profesora de matemáticas. Consiguió una exención, se graduó primera de su promoción y la mandaron a programar el Mark I en Harvard.

- En 1952 creó el **A-0**, lo que ella llamó el primer compilador, para que la computadora entendiera instrucciones legibles y no solo unos y ceros. Durante años nadie le hizo caso.
- La anécdota del primer **“bug”**: una polilla atrapada en un relé, pegada en el cuaderno de registro y que hoy está en el Smithsonian.

3. Nace COBOL (1959). Cada fabricante tenía su propio lenguaje y no eran compatibles, así que el comité CODASYL creó un lenguaje común para negocios. Lo hizo un comité, pero se basó en **FLOW-MATIC**, de Hopper, y en su idea central: código que se lea casi como inglés, entendible hasta por un contable. Se expandió rápido por bancos, gobiernos y aseguradoras.

4. Por qué lo odiaban. Precisamente por ser simple. Como lo entendía cualquiera, los académicos lo veían como algo menor: verboso y anticuado (se escribía `MULTIPLY` en vez de `*`). El video lo presenta como elitismo. Hopper siguió dando charlas contra la frase “siempre se ha hecho así”, llegó a contraalmirante y se retiró a los 79 años como la oficial en activo más veterana de EE. UU. Murió en 1992.

5. Los intentos de reemplazarlo:

- **PL/I** (IBM, años 60), que pretendía unir COBOL y Fortran y casi nadie recuerda.
- **Los lenguajes 4GL** (años 80), que prometían “programar sin programadores”.
- **Java** (años 90): los bancos lo adoptaron solo para la capa visible, y el motor que mueve el dinero siguió en COBOL.
- **La nube**, con consultoras que prometen la “migración total” sin que el núcleo se mueva.

6. Casos reales:

- **El sistema de impuestos de EE. UU. (IRS):** funciona desde ~1960 en COBOL y Assembly. Intentan reemplazarlo desde 2009 y la fecha prometida ahora es 2030.
- **Commonwealth Bank (Australia):** el presupuesto inicial de modernización fue de 580 millones y terminó superando los 1.000 millones en 5 años. Y ese es el caso *exitoso*.
- **TSB (Reino Unido, 2018):** migró su núcleo en un fin de semana. Casi 2 millones de clientes no pudieron entrar a sus cuentas (algunos veían el dinero de otros). Costó más de 330 millones de libras, perdió 80.000 clientes y el director renunció.

7. Por qué es intocable:

1. **Miles de millones de líneas** escritas durante 60 años, casi sin documentación.
2. **Cálculos exactos con decimales:** cuenta el dinero al centavo, mientras que los redondeos de otros lenguajes, multiplicados por billones de operaciones, se convierten en dinero perdido.
3. **Fiabilidad:** hay programas que llevan 40 años funcionando sin apagarse, y nadie quiere ser el que los toque.

Tu app bancaria moderna sería solo la "cara bonita"; por debajo sigue COBOL en un *mainframe*.

8. La bomba de tiempo. La mayoría de los programadores de COBOL tienen más de 55 años y casi no hay relevo. En 2020, en plena pandemia, las solicitudes de desempleo en Nueva Jersey subieron un 1.600 %. El sistema en COBOL colapsó y el gobernador Phil Murphy pidió voluntarios que supieran COBOL en televisión.

Idea central: COBOL no sobrevivió *a pesar* de ser simple y legible, sino *gracias* a eso: Hopper lo diseñó para que siguiera funcionando aunque quien lo escribió ya no estuviera. Hopper recibió la Medalla Presidencial de la Libertad en 2016. La ironía final es que ella luchó contra el "siempre se ha hecho así", y su lenguaje terminó siendo lo que nadie se atreve a cambiar.

Una nota: este video parece de **otro canal** (el estilo y la despedida son distintos a los de Archivo del Futuro). Al final anuncia un próximo video sobre otro lenguaje "que se le fue de las manos a su creador".

Este es el resumen de **"Cómo un lenguaje de SEGUNDA humilló a los de PRIMERA — Python"**. Es del mismo canal que el de COBOL y lo hice también con la transcripción completa.

1. El gancho. En 2013, en EuroPython (Florencia, más de 800 asistentes), se dio una charla titulada **"Why Python Sucks"** ("Por qué Python es una basura") y la sala aplaudió. La tesis del video es que Microsoft, las universidades y los profesores trataron a Python como un juguete durante años, y cuando se volvió el estándar nadie pidió perdón: solo sacaron cursos y ofertas de trabajo nuevos.

2. El origen (Navidad de 1989). Guido van Rossum, investigador del CWI en Ámsterdam, estaba aburrido en sus vacaciones y decidió escribir un intérprete como pasatiempo. Partió de **ABC**, un lenguaje educativo elegante pero que no se podía extender, y le agregó la posibilidad de extenderlo con C. Tomó tres decisiones que la industria consideró errores durante 25 años:

- **Indentación obligatoria** en lugar de llaves.
- **Tipado dinámico**: no hay que declarar los tipos.
- **Legibilidad antes que rendimiento.**

El nombre viene de *Monty Python*. Pensó que le llevaría una semana, pero la primera versión pública salió en febrero de 1991.

3. Los 22 años de desprecio. Los lenguajes "serios" eran C++, Java y Delphi. Python era para scripts y prototipos, no aparecía en las certificaciones y las universidades lo enseñaban solo como introducción. La ironía de la charla de 2013 es que la dio **Dmitry Jemerov, de JetBrains**, que trabajaba en Kotlin, un lenguaje rival. Guido nunca respondió a las críticas. Su convicción era que **el código se lee muchas más veces de las que se escribe**.

4. La adopción silenciosa:

- **Google:** Peter Norvig lo impulsó porque se programaba 5 a 10 veces más rápido que en C++, y el cuello de botella era el tiempo del programador, no el del procesador.
- **El ecosistema científico (2001–2006):** NumPy, SciPy y Matplotlib combinaron la legibilidad de Python con la velocidad de C y Fortran por debajo. Python le quitó terreno a MATLAB en las universidades.
- **Empresas:** YouTube, Reddit y Dropbox se construyeron en Python.

5. El machine learning. Entre 2007 y 2016 aparecieron Theano, Caffe, TensorFlow y PyTorch, de cuatro grupos distintos (Montreal, Berkeley, Google y Facebook), y todos eligieron Python sin ponerse de acuerdo. Para 2017 dominaba la IA académica.

6. El "cambio de bando" silencioso:

- En 2014 superó a Java como primer lenguaje que se enseña en las mejores universidades de EE. UU.
- Microsoft lo integró en Azure, GitHub y la extensión oficial de Python para VS Code. Google lo puso al frente de TensorFlow y Colab.
- Ninguna institución reconoció públicamente que se había equivocado.

7. La renuncia de Guido (julio de 2018). Dejó su puesto de "dictador benevolente vitalicio" después de 28 años. El detonante fue la pelea interna por el **operador morsa (`:=`)**, un cambio menor de sintaxis. Lo que nunca logró la industria en dos décadas, lo logró la propia comunidad. Años después dijo que debió soltar el control mucho antes.

8. El final irónico:

- En noviembre de 2020, **Microsoft contrató a Guido** para el proyecto *Faster CPython*. Él contó que aceptó porque se aburría jubilado.
- En agosto de 2023, Microsoft anunció **Python dentro de Excel**, que durante 30 años había funcionado con VBA.
- Hoy Guido sigue programando a los 70 años, pagado por Microsoft.

Idea central: Python ganó justamente por lo que lo hacía parecer "de segunda" (sencillo, legible, priorizando al programador), y lo hizo en silencio, sin que la industria admitiera que se equivocó.

Dos notas: el tono es más de opinión y provocación que el de los videos de Archivo del Futuro, y en la transcripción hay algunos datos confusos (por ejemplo, VS Code no fue "adquirido": es de Microsoft; lo que Microsoft adoptó fue la extensión de Python que había hecho la comunidad). Al final anuncia su video sobre Java, que según ellos le costó a Microsoft 1.600 millones de dólares. Esa cifra corresponde al acuerdo amplio con Sun de 2004, no al de 20 millones de 2001 que mencionaba el video de Java que viste antes.

Este es el resumen de **"Cómo un Ingeniero Creó a los Developers — C"**. Es del mismo canal que los de COBOL y Python, y lo hice con la transcripción completa.

1. El gancho. Los núcleos de Linux, macOS y Windows están escritos en **C**, un lenguaje de 1972. También muchas bases de datos, servidores y librerías críticas. El intérprete de Python (CPython) está escrito en C, y el motor de JavaScript (V8) en C++. Según el video, casi todo el "stack moderno" es una capa de pintura sobre C. Su creador, **Dennis Ritchie**, murió la misma semana que Steve Jobs, y casi nadie se enteró.

2. El fracaso de Multics. A finales de los 60, Ken Thompson y Dennis Ritchie llevaban 5 años en Bell Labs trabajando en Multics, un sistema operativo muy ambicioso. Bell Labs lo

canceló y prohibió intentar otro sistema operativo. Pero en el sótano había una máquina abandonada, una **PDP-7**.

3. Nace Unix (1969). La esposa de Thompson se fue un mes de visita con la familia, y él usó esas 4 semanas para escribir un sistema operativo nuevo, una pieza por semana. Brian Kernighan le puso el nombre de **Unix** como burla a Multics. Según Thompson, la motivación original era poder correr juegos.

4. El problema que vio Ritchie. Unix estaba escrito en Assembly, atado a esa máquina concreta. Cuando compraron una PDP-11 en 1970, había que reescribirlo todo. Ritchie propuso algo que en esa época sonaba absurdo: escribir el sistema operativo en un **lenguaje de alto nivel**. Tomó el lenguaje B de Thompson, le agregó tipos, estructuras y control de la memoria, y lo llamó **C**, la letra siguiente. En **1973 reescribieron Unix en C**: por primera vez un sistema operativo serio era portable, y el software podía sobrevivir al hardware.

5. El accidente legal que lo expandió. AT&T, la empresa dueña de Bell Labs, era el monopolio telefónico de EE. UU. y tenía prohibido por orden judicial vender software. Por eso **regaló Unix a las universidades casi gratis, con código fuente incluido**, "tal como está, sin soporte". En pocos años estaba en todos los laboratorios, y una generación entera aprendió a programar en C.

6. El libro "K&R" (1978). *El lenguaje de programación C*, de Kernighan y Ritchie, tenía solo 228 páginas cuando lo normal eran 600 o más. Ahí apareció el famoso **"Hello, World"**, que desde entonces es el primer programa que escribe cualquiera que aprende un lenguaje.

7. C se vuelve el suelo:

- **Linux** (1991, Linus Torvalds, 21 años) se escribió en C.
- **Windows NT** (1993), la base del Windows 11 actual, también.
- **NeXTSTEP** se convirtió en la base de macOS y luego de iOS.

Todos los lenguajes que se vendieron como "olvídate de C" terminaron funcionando sobre C por dentro. C no fue reemplazado: pasó a ser infraestructura invisible.

8. Dos muertes, una semana. Steve Jobs murió el **5 de octubre de 2011**, con portadas y homenajes en todo el mundo. Ritchie fue **encontrado muerto en su casa el 12 de octubre**, a los 70 años. La noticia la dio días después su excolega Rob Pike en Google+. La simetría que señala el video: sin Ritchie no hay Unix; sin Unix no hay NeXT; sin NeXT no hay Mac OS X ni iPhone.

Idea central: Ritchie no tiene monumentos, pero su "gramática" (las llaves, los tipos, el `return`, el `printf`) la heredaron C++, Java, JavaScript, Go y Rust. Su legado aparece cada vez que alguien escribe su primera línea de código.

Al final anuncia su video sobre Linux. Un detalle: el "hello, world" ya aparecía en un tutorial interno de Kernighan en Bell Labs de 1972, pero fue el libro de 1978 el que lo hizo famoso.

Este es el resumen de **"Por qué es IMPOSIBLE matar a FORTRAN"**. Es del mismo canal que los de COBOL, Python y C (de hecho, tiene casi la misma estructura que el de COBOL), y lo hice con la transcripción completa.

1. El gancho. John Backus, ganador del premio Turing, creó Fortran en 1957 y años después **renegó de su propia creación**: dijo que los lenguajes como el suyo estaban "rotos por dentro". Aun así, en los centros de supercomputación la mayoría de los proyectos (el video dice 6 de cada 10) siguen en Fortran. Con él se pronostica el clima y se diseñan aviones.

2. El problema de los años 50. Programar era escribir a mano, número por número, en el idioma de la máquina. La **IBM 704** costaba millones, y lo caro no era la máquina sino el tiempo de programarla: semanas por programa, y un solo error lo arruinaba todo.

3. La idea "para vagos". Backus propuso escribir los cálculos como fórmulas matemáticas y que un traductor (un compilador) los convirtiera al idioma de la máquina. La élite dijo que era imposible, porque un traductor nunca igualaría a un experto. El propio Backus admitió que lo hizo porque **odiaba el trabajo pesado**. Mandó el memo en 1953 y su equipo trabajó en ello desde 1954.

4. El éxito (1957). Lograron el **primer compilador optimizador**, con código casi tan rápido como el escrito a mano. IBM lo incluyó en cada máquina que vendía, y en pocos años la ciencia y la ingeniería de todo el mundo programaban en Fortran.

5. El desprecio:

- **Los académicos:** era demasiado fácil y su `GOTO` producía "código espagueti".

- **Dijkstra (1975):** lo describió como un niño que nunca maduró, algo que nadie serio debería usar.
- **El golpe más duro vino del propio Backus**, en su discurso del premio Turing de 1977: dijo que estos lenguajes eran "gordos y débiles", defectuosos de raíz. Pasó el resto de su carrera buscando una alternativa (la programación funcional, para escapar del "cuello de botella de von Neumann").

Durante décadas, saber Fortran se veía como "ser de los antiguos".

6. Los que intentaron reemplazarlo. C, C++ y, más recientemente, **Julia**. El caso más irónico es **Python**: domina la ciencia y la IA, pero cuando hay que procesar números en serio, muchas veces llama por debajo a rutinas escritas en Fortran.

7. Por qué es imposible de matar:

1. Fue hecho para una sola cosa, **procesar números y matrices a máxima velocidad**, y lo sigue haciendo mejor que casi cualquier otro.
2. Hay una montaña de código científico **verificado durante décadas**.
3. **Reproducibilidad:** si reescribes una simulación del clima y da un número distinto, no sabes si descubriste algo o rompiste algo. Por eso nadie toca rutinas de hace 40 años.

8. Dónde se usa hoy:

- **Fugaku**, la supercomputadora japonesa que fue la más rápida del mundo.
- **Lawrence Livermore**, donde simulan el arsenal nuclear de EE. UU. (no hacen pruebas reales desde 1992).
- **Los modelos del clima y de huracanes**, con cientos de miles de líneas en Fortran.

9. Está vivo, no solo sobrevive. Tiene un estándar nuevo de **2023**, una comunidad de "Fortran moderno" y soporte para repartir el trabajo entre miles de procesadores. Hay jóvenes que lo aprenden a propósito para simulaciones científicas.

Idea central: igual que COBOL, Fortran sobrevivió *gracias* a lo que la élite despreciaba: ser estrecho, terco y especializado. La ironía final es que Backus fracasó con el lenguaje que iba a reemplazarlo. Cuando murió, en 2007, todos los titulares decían "murió el creador de Fortran", justo lo que él había intentado dejar atrás.

Este es el resumen de **"Por qué es casi imposible hacer luz LED azul"**. Lo hice con la transcripción completa. Parece la versión en español de un video de Veritasium, y es de otro tema: física, no programación.

1. El problema. El color de un LED no viene de su plástico, sino del material semiconductor. El primer LED visible (Nick Holonyak, General Electric, 1962) era **rojo**, y luego llegó el **verde**. Faltaba el **azul**: con rojo, verde y azul se puede hacer luz blanca y cualquier color, es decir, iluminación, pantallas y televisores. Empresas como IBM, GE, Bell Labs, Sony, Toshiba y Panasonic lo intentaron durante 30 años, gastando cientos de millones, sin éxito.

2. Shuji Nakamura y Nichia. Nakamura trabajaba en **Nichia**, una pequeña química japonesa cuya división de semiconductores estaba fracasando. Su laboratorio lo había armado él mismo, y las fugas de fósforo causaban explosiones. En 1988 le pidieron la renuncia. Desesperado, le propuso al fundador, **Nobuo Ogawa**, buscar el LED azul, y Ogawa le dio **500 millones de yenes (~3 millones de dólares)**, alrededor del 15 % de las utilidades de la empresa.

3. La física, en resumen. Un LED une un semiconductor **tipo n** (con electrones de sobra) y uno **tipo p** (con "huecos"). Cuando un electrón cae en un hueco, suelta su energía como un **fotón**, y el tamaño de esa brecha de energía define el color: la del silicio da infrarrojo (controles remotos), y para el azul se necesita una brecha más grande. Por eso el orden fue rojo, verde y por último azul. Además, el cristal tiene que ser casi perfecto, porque cada defecto convierte la energía en calor en vez de luz.

4. Los tres obstáculos que superó Nakamura:

1. **Un cristal de alta calidad.** Pasó un año en Florida aprendiendo la técnica **MOCVD**; lo menospreciaban por no tener doctorado. En Japón podía obtenerlo publicando 5 artículos, así que eligió el **nitruro de galio**, que casi todos habían abandonado (en un congreso, el seleniuro de zinc tenía 500 asistentes y el nitruro de galio, 5). En 1990 inventó el **reactor de doble flujo**: una segunda boquilla de gas que "aplasta" los gases contra la base y forma un cristal uniforme.
2. **El nitruro de galio tipo p.** Akasaki y Amano (Universidad de Nagoya) ya lo habían logrado con haces de electrones, pero sin saber por qué. Nakamura descubrió que bastaba **calentarlo a 400 °C**: el hidrógeno del amoníaco bloqueaba los huecos, y el calor lo liberaba. El método era rápido y servía para producir en masa.
3. **La potencia.** Su prototipo daba solo 42 microvatios y era violeta. Agregó una capa de **nitruro de galio e indio**, que daba un azul de verdad y más eficiencia (todos decían que esos materiales no se mezclaban), y otra de **nitruro de aluminio y galio** para que los electrones no se escaparan.

Mientras tanto, el nuevo presidente, **Eiji Ogawa** (yerno del fundador), le ordenaba por escrito que dejara de hacerlo. Nakamura **arrugaba las notas y las tiraba**, y publicaba sin permiso.

5. El triunfo. Logró un LED **azul puro (450 nm)**, más de 100 veces más brillante que los anteriores y visible a la luz del día. Nichia lo anunció en Tokio y la industria quedó atónita. Para 1994 fabricaban un millón al mes. En **1996** sacaron el **LED blanco**, con un fósforo amarillo encima del azul, y para 2001 Nichia facturaba unos 700 millones de dólares al año.

6. La injusticia. Nakamura recibió **unos 180 dólares por patente** y un aumento de sueldo. Se fue a EE. UU. en 2000, Nichia lo demandó y él contrademandó. Un tribunal japonés le dio la razón, pero al final se resolvió con **unos 8 millones de dólares**, que apenas cubrieron sus gastos legales, por una industria que hoy vale unos **80.000 millones**.

7. El impacto. Los LED son mucho más eficientes que los focos incandescentes (que producen sobre todo calor), duran más y el costo se recupera en unos dos meses. En 2010 eran el 1 % de las ventas de iluminación doméstica; en 2022, más de la mitad. Usarlos en todas partes ahorraría millones de toneladas de CO_2.

8. El final. En **2014**, Nakamura, Akasaki y Amano ganaron el **Nobel de Física**. Nakamura le ofreció a Nichia hacer las paces y la empresa se negó. Obtuvo su doctorado y hoy tiene más de 900 artículos publicados. Investiga micro-LEDs (para realidad virtual y aumentada), LEDs ultravioleta para esterilizar y hasta fusión nuclear. ¿Su color favorito? El azul, por el mar frente a su casa de niño.

Una nota: en la transcripción automática algunas cifras salen confusas (por ejemplo, la potencia final del LED aparece como "15 microvatios", cuando tenía que superar los 1.000), así que puse las más coherentes con el resto del video.

Este es el resumen de **"¿¿¿El PROGRAMADOR más importante de la HISTORIA???"**, sobre **Linus Torvalds**. Es del mismo canal que los de COBOL, Python, C y Fortran, y lo hice con la transcripción completa.

1. El gancho. Hay quien lo considera el programador más importante de la historia: Linux está en tu celular, en los servidores de tu banco y en la nube. Pero él mismo se describe como alguien desagradable. El video arranca con su famoso insulto a NVIDIA frente a una sala de estudiantes.

2. Infancia en Helsinki. Nació a finales de 1969 en una familia de periodistas y políticos (su padre, comunista, llegó a eurodiputado), gente que vivía de hablar, y él era el callado. Sus padres se separaron y lo criaron su madre y su abuelo Leo, profesor de estadística, que le regaló un **Commodore VIC-20**. A los 11 años tecleaba los cálculos del abuelo y

descubrió que una máquina hace exactamente lo que le dices. Programaba por diversión, sin plan, y creció rechazando todo lo que sonara a ideología: solo quería que las cosas funcionaran.

3. El nacimiento de Linux (1991). Con 21 años, estudiando en la Universidad de Helsinki, le frustraba **Minix**, así que se compró un 386 y se encerró meses a programar su propio sistema. El 26 de agosto de 1991 anunció en un foro que era "solo un hobby, nada grande ni profesional como GNU". Ese **GNU** era el proyecto de **Richard Stallman**, al que le faltaba justo el núcleo, y Linus lo había escrito sin buscarlo. Lo **regaló** y usó la licencia de Stallman, la GPL, no por ideología sino porque le servía.

4. Contra Tanenbaum (1992). Andrew Tanenbaum, el profesor más respetado en sistemas operativos, publicó "**Linux está obsoleto**" y dijo que con ese diseño no le habría puesto buena nota. Linus, un estudiante desconocido, le respondió de igual a igual. Treinta años después, ese sistema "obsoleto" sostiene el planeta.

5. Anécdotas:

- **Su esposa:** en 1993 pidió a sus alumnos un email de prueba, y una de ellas, **Tove Monni** (seis veces campeona de karate de Finlandia), le respondió pidiéndole una cita. Siguen casados.
- **El pingüino:** le mordió uno en un zoológico de Australia y le pareció tan gracioso que lo hizo el logo de Linux (Tux).

6. Millonario sin cobrar. En 1997 se mudó a California a trabajar en Transmeta con un sueldo normal, en plena fiebre de las puntocom. Red Hat y VA Linux le regalaron acciones, y cuando salieron a bolsa en 1999 valían unos **20 millones de dólares**. Hoy la Linux Foundation, financiada por los gigantes que dependen de su código, le paga más de un millón al año. Tiene tres hijas: Patricia, Daniela y Celeste.

7. Git (2005). La empresa dueña de la herramienta que usaba Linux para coordinar a miles de programadores se la quitó. Linus escribió **Git** desde cero en unas dos semanas, y hoy lo usan prácticamente todos los desarrolladores del mundo. Hoy Linux está en el 100 % de las 500 supercomputadoras más potentes, en la mayoría de los celulares (Android), en la nube, en la bolsa de Nueva York y en la NASA. Microsoft, que en 2001 lo llamó "un cáncer", hoy incluye Linux dentro de Windows y dice que lo "ama".

8. El lado oscuro:

- El "¡que te j***, NVIDIA!" de 2012.
- Correos humillantes a colaboradores ("no sé cómo llegó a adulto siendo tan idiota").
- Frases como "no soy una persona agradable y me dan igual; me importa el kernel".
- **Sarah Sharp**, una desarrolladora muy respetada del kernel, le pidió en 2013 que dejara de maltratar a la gente. Se fue del proyecto en 2015 por el trato abusivo.

9. El contraste. Escondió las **fechas de nacimiento de sus hijas en hexadecimal** dentro del código del kernel. El video lo interpreta como alguien que "solo sabe querer a través de una máquina", lo que hace más grave el daño a los demás, porque sí sabía sentir.

10. El cambio (2018). Pidió perdón públicamente ("no soy una persona emocionalmente empática"), se apartó un tiempo del proyecto y se puso a aprender a tratar bien a la gente. Hoy vive en Portland, trabaja desde casa y habla con mucha más calma.

Conclusión del video: por escala, nadie lo supera; lo de "el más importante" lo deja abierto. Menciona como candidatos a **Dennis Ritchie** (sin C ni Unix no habría Linux), Turing y Berners-Lee. Lo único que lo distingue, según el video, es que cambió el mundo dos veces (con Linux y con Git) por pura diversión. Lo resume con su frase: "No soy un visionario, soy un ingeniero: miro al suelo y quiero arreglar el bache que tengo delante".